

```
In [1]: import torch
import torchvision
from torch import nn # neural network: 包含搭建模型常用发层和工具
from torch.utils import data
from torchvision import transforms
import matplotlib.pyplot as plt
```

```
In [2]: def get_dataloader_workers():
        """使用 4 个子进程并行读取数据。"""
        return 4
```

```
In [3]: def load_data_fashion_mnist(batch_size, resize=None): # resize=None 表示是否调整图片大小
        """下载Fashion-MNIST数据集, 然后将其加载到内存中"""
        trans = [transforms.ToTensor()]
        if resize:
            trans.insert(0, transforms.Resize(resize)) # 在转换为 tensor 前先调整大小
        trans = transforms.Compose(trans) # 组合所有的操作
        mnist_train = torchvision.datasets.FashionMNIST(
            root="./data", train=True, transform=trans, download=True)
        mnist_test = torchvision.datasets.FashionMNIST(
            root="./data", train=False, transform=trans, download=True)
        return (data.DataLoader(mnist_train, batch_size, shuffle=True,
                                num_workers=get_dataloader_workers()),
                data.DataLoader(mnist_test, batch_size, shuffle=False,
                                num_workers=get_dataloader_workers()))
```

```
In [4]: batch_size = 256
train_iter, test_iter = load_data_fashion_mnist(batch_size)
```

4.6.1. 初始化模型参数

```
In [5]: num_inputs, num_outputs, num_hiddens1, num_hiddens2 = 784, 10, 256, 256
```

4.6.2. 定义 dropout 函数

```
In [6]: def dropout_layer(X, dropout):
        assert 0 <= dropout <= 1 # 要求 dropout 必须在 0 到 1 之间 (含边界)
        if dropout == 1: # 在本情况中, 所有元素都被丢弃
            return torch.zeros_like(X)
        if dropout == 0: # 在本情况中, 所有元素都被保留
            return X
        mask = (torch.rand(X.shape) > dropout).float()
        # torch.rand(X.shape): 创建一个与 X 同形状的张量, 其中每个元素都是 [0, 1) 的随机数
        # .float(): 将布尔值转换为浮点数 (True -> 1.0, False -> 0.0)
        return mask * X / (1.0 - dropout)
```

4.6.3. 定义模型

```
In [7]: dropout1, dropout2 = 0.2, 0.5
```

```
class Net(nn.Module):
    def __init__(self, num_inputs, num_outputs, num_hiddens1, num_hiddens2,
                  is_training = True):
        super(Net, self).__init__() # 调用父类 nn.Module 的构造函数, 获得 PyTorch 模型应有的能力
        self.num_inputs = num_inputs
        self.training = is_training
        # 创建三个线性层 (全连接层), 以及一个 ReLU 激活函数
        self.lin1 = nn.Linear(num_inputs, num_hiddens1)
        self.lin2 = nn.Linear(num_hiddens1, num_hiddens2)
        self.lin3 = nn.Linear(num_hiddens2, num_outputs)
        self.relu = nn.ReLU()

    def forward(self, X):
        H1 = self.relu(self.lin1(X.reshape((-1, self.num_inputs))))
        # 只有在训练模型时才使用dropout
        if self.training == True: # 在第一个全连接层之后添加一个dropout层
            H1 = dropout_layer(H1, dropout1)
        H2 = self.relu(self.lin2(H1))
        if self.training == True: # 在第二个全连接层之后添加一个dropout层
            H2 = dropout_layer(H2, dropout2)
        out = self.lin3(H2)
        return out

net = Net(num_inputs, num_outputs, num_hiddens1, num_hiddens2)
```

4.6.4. 损失函数

```
In [8]: loss = nn.CrossEntropyLoss(reduction='none')
# reduction='none': 不把一个批次中所有样本的损失汇总为一个数, 而是保留每个样本各自的损失。
```

4.6.5. 训练和测试

```
In [9]: def accuracy(y_hat, y):
    """计算预测正确的样本数。"""
    # .item(): 把“只包含一个值的 PyTorch 张量转换为普通 Python 数值。
    return (y_hat.argmax(dim=1) == y).sum().item()

def evaluate_accuracy(net, data_iter):
    """计算模型在数据集上的准确率。"""
    correct, total = 0, 0
    with torch.no_grad():
        for X, y in data_iter:
            correct += accuracy(net(X), y)
            total += y.numel()
    return correct / total

def train(net, train_iter, test_iter, loss, num_epochs, optimizer):
    """训练模型, 并在每个迭代周期后输出训练损失和准确率。"""
    device = next(net.parameters()).device # 获取模型参数所在的设备 (CPU 或 GPU)
    for epoch in range(num_epochs):
        net.train() # 启用 nn.Module 的训练模式, 使 Dropout 生效
        total_loss, correct = 0.0, 0, 0
        for X, y in train_iter:
            X, y = X.to(device), y.to(device) # 将数据和标签移动到模型所在的设备上
            optimizer.zero_grad()
            y_hat = net(X)
            l = loss(y_hat, y).mean()
            l.backward()
            optimizer.step()

            total_loss += l.item() * y.numel()
            correct += accuracy(y_hat, y)
            total += y.numel()

        train_loss = total_loss / total
        train_acc = correct / total
        test_acc = evaluate_accuracy(net, test_iter)
        print(f'epoch {epoch + 1}: loss {train_loss:.4f}, train acc {train_acc:.3f}, test acc {test_acc:.3f}')
        net.eval() # 训练结束后关闭 Dropout, 便于后续预测
```

```
In [10]: num_epochs, lr, batch_size = 10, 0.5, 256
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

```
train(net, train_iter, test_iter, loss, num_epochs, optimizer)
```

```
epoch 1: loss 0.9008, train acc 0.667, test acc 0.709
epoch 2: loss 0.5312, train acc 0.804, test acc 0.798
epoch 3: loss 0.4604, train acc 0.831, test acc 0.833
epoch 4: loss 0.4268, train acc 0.844, test acc 0.822
epoch 5: loss 0.4000, train acc 0.853, test acc 0.800
epoch 6: loss 0.3883, train acc 0.859, test acc 0.839
epoch 7: loss 0.3696, train acc 0.865, test acc 0.830
epoch 8: loss 0.3561, train acc 0.870, test acc 0.819
epoch 9: loss 0.3438, train acc 0.874, test acc 0.850
epoch 10: loss 0.3339, train acc 0.878, test acc 0.858
```

4.6.6. 预测

```
In [11]: def get_fashion_mnist_labels(labels):
        """返回Fashion-MNIST数据集的文本标签"""
        text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
                        'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
        return [text_labels[int(i)] for i in labels]
```

```
In [12]: def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5): #@save
        """绘制图像列表"""
        figsize = (num_cols * scale, num_rows * scale) # 计算画布的大小
        _, axes = plt.subplots(num_rows, num_cols, figsize=figsize) # 创建多个图片窗口, axes 表示所有子图对象
        axes = axes.flatten() # 将 axes 展平成一维数组
        for i, (ax, img) in enumerate(zip(axes, imgs)): # zip() 实现配对得到 (ax, img); enumerate() 给列表增加索引
            if torch.is_tensor(img):
                # 图片张量
                ax.imshow(img.numpy()) # matplotlib 只认识 numpy, 不认识 tensor
            else:
                # PIL 图片
                ax.imshow(img)
            # 隐藏坐标轴
            ax.axes.get_xaxis().set_visible(False)
            ax.axes.get_yaxis().set_visible(False)
            # 显示标题
            if titles:
                ax.set_title(titles[i])
        return axes
```

```
In [13]: def predict_ch3(net, test_iter, n=6):
        """预测标签"""
        for X, y in test_iter: # 取一个batch
            break
        trues = get_fashion_mnist_labels(y) # 真实标签
        preds = get_fashion_mnist_labels(net(X).argmax(axis=1)) # 预测标签
        titles = [true + '\n' + pred for true, pred in zip(trues, preds)] # 标题
        show_images(X[:n].reshape(n, 28, 28), 1, n, titles=titles[:n]) # 显示图片

predict_ch3(net, test_iter)
```

